Hospital Readmission Risk Predictor — Notebook 5: MLflow Registration & FastAPI Serving

Prepared for: Hospital Administration & Care Management Stakeholders **Prepared by:** Angeline Setiawan | **Team:** Epsilon **Project:** Hospital Readmission Risk Predictor — A Phased MLOps Project

Purpose

This notebook completes the Phase 1 required build by taking the tuned classifier selected in `04_Hyperparameter_Tuning_Model_Evaluation_V5.ipynb` and turning it into something that can actually be run as a system rather than a notebook result:

1. **Log the selected model as a tracked MLflow run** — parameters, test-set metrics, and the fitted pipeline itself, so the decision that led to this model is reproducible and auditable later.
2. **Register the model and manually promote it to "Production"** in the MLflow Model Registry, matching the promotion step described in the project proposal.
3. **Build a FastAPI endpoint** that always loads whichever model version is currently tagged "Production" and serves live predictions from it, at the classification threshold that was tuned for that specific model.

Why this matters, and not just as a formality: a model sitting in a notebook cannot be called by a downstream system, cannot be compared against a future challenger model in a reproducible way, and has no record of exactly which data, parameters, and code produced it. Logging to MLflow and serving through FastAPI is what turns "a model that scored well in a notebook" into "a system a hospital's care-management workflow could actually call." It is also the foundation Phase 2 (automated Airflow retraining) will sit on top of: Phase 2 only has to add a retrain-evaluate-promote loop around the same registry and tagging pattern built here, without touching the FastAPI layer at all.

This notebook does not retrain anything. It loads the exact fitted pipeline saved at the end of Notebook 4 (`outputs/tuned_supervised_model.joblib`) and the exact train/test split used to fit it, so the model registered here is identical to the one already evaluated and reported on in the Hyperparameter Tuning & Model Evaluation report.

1. Imports and Configuration

In [1]:
```
# ---------------------------------------------------------------------
# 1.1 Standard imports
# ---------------------------------------------------------------------
import os
import json
import joblib
import numpy as np
import pandas as pd
from pathlib import Path

# ---------------------------------------------------------------------
# 1.2 MLflow imports
# ---------------------------------------------------------------------
# MLflow is a required dependency for this notebook (unlike xgboost in
# Notebook 4, there is no fallback path -- registering and serving a
# "Production" model is the whole point of this notebook).
try:
    import mlflow
    import mlflow.sklearn
    from mlflow.tracking import MlflowClient
except ImportError as exc:
    raise ImportError(
        "This notebook requires mlflow. Run `pip install mlflow` and re-run this cell."
    ) from exc

# ---------------------------------------------------------------------
# 1.3 FastAPI / serving imports
# ---------------------------------------------------------------------
# These are only needed for Section 9 (writing the serving app) and
# Section 10 (smoke-testing it in-notebook). Imported up front so a
# missing dependency fails fast rather than partway through the notebook.
try:
    import fastapi  # noqa: F401
    from fastapi.testclient import TestClient
except ImportError as exc:
    raise ImportError(
        "This notebook requires fastapi (and httpx, its TestClient dependency). "
        "Run `pip install fastapi httpx uvicorn` and re-run this cell."
    ) from exc
```

```
RANDOM_STATE = 42

print("Imports loaded.")
```

```
C:\Users\angie\anaconda3\envs\pytorch\lib\site-packages\tqdm\auto.py:21: TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
Imports loaded.
C:\Users\angie\anaconda3\envs\pytorch\lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
  from starlette.testclient import TestClient as TestClient  # noqa
```

2. Load Phase 1 Artifacts

Everything loaded here was produced by `04_Hyperparameter_Tuning_Model_Evaluation_V5.ipynb`: the fitted pipeline, the processed train/test tables it was fit on, and the summary tables it wrote out. Nothing is recomputed from raw data in this notebook -- the point is to register and serve the exact model that was already evaluated and reported on, not a fresh copy of it.

In [2]:
```
# ------------------------------------------------------------------------
# 2.1 Locate the outputs and processed-data directories
# ------------------------------------------------------------------------
def first_existing(paths):
    for path in paths:
        if os.path.exists(path):
            return path
    return None

OUTPUT_DIR = first_existing(["../outputs", "outputs"])
if OUTPUT_DIR is None:
    raise FileNotFoundError(
        "Could not find the outputs/ directory from Notebook 4. "
        "Run 04_Hyperparameter_Tuning_Model_Evaluation_V5.ipynb first."
    )

TRAIN_PATH = first_existing(["../data/processed/train.csv", "data/processed/train.csv"])
TEST_PATH = first_existing(["../data/processed/test.csv", "data/processed/test.csv"])
if TRAIN_PATH is None or TEST_PATH is None:
    raise FileNotFoundError(
        "Could not find train.csv/test.csv in data/processed/. "
        "Update TRAIN_PATH/TEST_PATH if your repository layout differs."
    )

print(f"Using outputs directory: {OUTPUT_DIR}")
print(f"Using train/test data:   {TRAIN_PATH}, {TEST_PATH}")

# ------------------------------------------------------------------------
# 2.2 Load the fitted pipeline saved at the end of Notebook 4
# ------------------------------------------------------------------------
tuned_model = joblib.load(os.path.join(OUTPUT_DIR, "tuned_supervised_model.joblib"))
print(f"Loaded fitted pipeline: {tuned_model.named_steps['model'].__class__.__name__}")

# ------------------------------------------------------------------------
# 2.3 Load the summary tables Notebook 4 wrote out
# ------------------------------------------------------------------------
model_comparison = pd.read_csv(os.path.join(OUTPUT_DIR, "model_family_comparison.csv"))
tuned_test_metrics = pd.read_csv(os.path.join(OUTPUT_DIR, "tuned_test_metrics.csv"), index_col=0)

# ------------------------------------------------------------------------
# 2.4 Rebuild X_test / y_test using the same column logic as Notebook 4
# ------------------------------------------------------------------------
# This exclusion list must stay identical to Notebook 4, Section 3.1, since
# the loaded pipeline was fit on exactly these columns in this order.
TARGET = "readmitted_30d"
EXCLUDED_COLUMNS = (
    ["patient_nbr", "encounter_id", "encounter_order"]
    + ["diag_1", "diag_2", "diag_3"]
    + ["discharge_disposition_id", "discharge_disposition_id_label"]
    + [TARGET]
)

test_df = pd.read_csv(TEST_PATH)
FEATURE_COLUMNS = [c for c in test_df.columns if c not in EXCLUDED_COLUMNS]

X_test = test_df[FEATURE_COLUMNS].copy()
y_test = test_df[TARGET].astype(int).copy()

print(f"X_test shape: {X_test.shape}")
print(f"Modeling feature count: {len(FEATURE_COLUMNS)}")
```

```
Using outputs directory: outputs
Using train/test data:   ../data/processed/train.csv, ../data/processed/test.csv
Loaded fitted pipeline: LogisticRegression
X_test shape: (17610, 49)
Modeling feature count: 49
```

3. Confirm the Selected Model and Threshold

In [3]:
```python
# ----------------------------------------------------------------------
# 3.1 Selected model family and its tuned hyperparameters
# ----------------------------------------------------------------------
# model_comparison is already sorted best-CV-PR-AUC-first by Notebook 4,
# so row 0 is the family that was selected and saved to tuned_model.
best_model_name = model_comparison.loc[0, "Model Family"]
best_cv_pr_auc = float(model_comparison.loc[0, "Best CV PR-AUC"])

# Pulled directly from the fitted pipeline's model step rather than from a
# saved search object, since only the fitted pipeline itself was saved.
raw_params = tuned_model.named_steps["model"].get_params()

# Keep only JSON/MLflow-friendly scalar values; drop anything that is not
# a plain str/int/float/bool/None (e.g. nested estimator objects), and
# stringify the rest so mlflow.log_params doesn't choke on non-string values.
model_hyperparams = {
    k: (v if isinstance(v, (str, int, float, bool)) or v is None else str(v))
    for k, v in raw_params.items()
}

# ----------------------------------------------------------------------
# 3.2 Selected classification threshold
# ----------------------------------------------------------------------
# Read from Notebook 4's saved metrics table rather than hardcoded here,
# so this notebook always logs whatever threshold was actually selected,
# even if Notebook 4 is re-run later with a different result.
SELECTED_THRESHOLD = float(tuned_test_metrics.loc["Selected (tuned)", "Threshold"])

print(f"Selected model family:  {best_model_name}")
print(f"Best CV PR-AUC:         {best_cv_pr_auc:.4f}")
print(f"Selected threshold:     {SELECTED_THRESHOLD:.2f}")
print(f"Hyperparameters logged: {len(model_hyperparams)} values")
```

```
Selected model family:  Logistic Regression (tuned)
Best CV PR-AUC:         0.1603
Selected threshold:     0.43
Hyperparameters logged: 15 values
```

4. Recompute Final Test Metrics for Logging

These metrics are recomputed here from `tuned_model` and `X_test` / `y_test` directly, rather than copied from the Notebook 4 CSV, so the numbers written to MLflow are generated in this run and are guaranteed to match the model artifact being logged in the same run.

In [4]:
```python
# ----------------------------------------------------------------------
# 4.1 Score the test set at the selected threshold
# ----------------------------------------------------------------------
from sklearn.metrics import (
    average_precision_score,
    roc_auc_score,
    precision_score,
    recall_score,
    f1_score,
    accuracy_score,
)

y_test_prob = tuned_model.predict_proba(X_test)[:, 1]
y_test_pred = (y_test_prob >= SELECTED_THRESHOLD).astype(int)

final_metrics = {
    "test_pr_auc": average_precision_score(y_test, y_test_prob),
    "test_roc_auc": roc_auc_score(y_test, y_test_prob),
    "test_precision": precision_score(y_test, y_test_pred, zero_division=0),
    "test_recall": recall_score(y_test, y_test_pred, zero_division=0),
    "test_f1": f1_score(y_test, y_test_pred, zero_division=0),
    "test_accuracy": accuracy_score(y_test, y_test_pred),
    "cv_pr_auc": best_cv_pr_auc,
}

for name, value in final_metrics.items():
    print(f"{name:15s}: {value:.4f}")
```

```
test_pr_auc    : 0.1086
test_roc_auc   : 0.6080
test_precision : 0.0804
test_recall    : 0.7868
test_f1        : 0.1459
test_accuracy  : 0.3715
cv_pr_auc      : 0.1603
```

5. Configure MLflow Tracking

In [5]:
```
# # ---------------------------------------------------------------------
# # 5.1 Point MLflow at a local, file-based tracking store
# # ---------------------------------------------------------------------
# # A local "mlruns/" folder is sufficient for this phase: it gives full
# # experiment tracking and a working Model Registry without standing up a
# # separate tracking server. If this project moves to a shared team setup
# # later, this is the one line that changes -- point MLFLOW_TRACKING_URI at
# # a remote server instead, and everything else in this notebook is unchanged.
# MLFLOW_TRACKING_URI = "file:./mlruns"
# mlflow.set_tracking_uri(MLFLOW_TRACKING_URI)

# EXPERIMENT_NAME = "hospital-readmission-risk"
# MODEL_NAME = "hospital-readmission-risk-classifier"

# mlflow.set_experiment(EXPERIMENT_NAME)

# print(f"Tracking URI: {MLFLOW_TRACKING_URI}")
# print(f"Experiment:   {EXPERIMENT_NAME}")
# print(f"Registry name for the model: {MODEL_NAME}")
```

In [8]:
```
# ---------------------------------------------------------------------
# MLflow configuration
# ---------------------------------------------------------------------

from pathlib import Path
import mlflow

# Notebook is located in:
# project_root/notebooks/
# Therefore, .parent points to the project root.
PROJECT_ROOT = Path.cwd().parent

# Use the single MLflow database stored at the project root.
MLFLOW_DB = PROJECT_ROOT / "mlflow.db"
MLFLOW_TRACKING_URI = f"sqlite:///{MLFLOW_DB.as_posix()}"

mlflow.set_tracking_uri(MLFLOW_TRACKING_URI)

EXPERIMENT_NAME = "hospital-readmission-risk"
MODEL_NAME = "hospital-readmission-risk-classifier"

mlflow.set_experiment(EXPERIMENT_NAME)

print(f"Project root:    {PROJECT_ROOT}")
print(f"MLflow database: {MLFLOW_DB}")
print(f"Tracking URI:    {MLFLOW_TRACKING_URI}")
print(f"Experiment:      {EXPERIMENT_NAME}")
print(f"Model name:      {MODEL_NAME}")
```

```
2026/08/24 13:42:11 INFO mlflow.tracking.fluent: Experiment with name 'hospital-readmission-risk' does not exist. Creating a new experiment.
Project root:    C:\Users\angie\Documents\GitHub\hospital-readmission-risk-predictor\AngelineSetiawan-hospital-readmission-risk-predictor
MLflow database: C:\Users\angie\Documents\GitHub\hospital-readmission-risk-predictor\AngelineSetiawan-hospital-readmission-risk-predictor\mlflow.db
Tracking URI:    sqlite:///C:/Users/angie/Documents/GitHub/hospital-readmission-risk-predictor/AngelineSetiawan-hospital-readmission-risk-predictor/mlflow.db
Experiment:      hospital-readmission-risk
Model name:      hospital-readmission-risk-classifier
```

In [9]:
```
variables_to_check = [
    "tuned_model",
    "SELECTED_THRESHOLD",
    "final_metrics",
    "best_model_name",
    "model_hyperparams",
    "FEATURE_COLUMNS",
]
```

```
for var in variables_to_check:
    print(f"{var}: {'DEFINED' if var in globals() else 'NOT DEFINED'}")
```

```
tuned_model: DEFINED
SELECTED_THRESHOLD: DEFINED
final_metrics: DEFINED
best_model_name: DEFINED
model_hyperparams: DEFINED
FEATURE_COLUMNS: DEFINED
```

6. Log the Run: Parameters, Metrics, and the Model Artifact

In [10]:
```
# ---------------------------------------------------------------------
# 6.1 Log everything in a single run and register the model in one step
# ---------------------------------------------------------------------
# mlflow.sklearn.log_model(..., registered_model_name=MODEL_NAME) both logs
# the fitted pipeline as a run artifact AND creates (or adds a new version
# to) a registered model in the Model Registry, so no separate registration
# call is needed.
RUN_NAME = f"{best_model_name.split(' ')[0].lower()}-threshold-{SELECTED_THRESHOLD:.2f}"

with mlflow.start_run(run_name=RUN_NAME) as run:
    RUN_ID = run.info.run_id

    # Parameters: enough to reconstruct why this specific model was chosen.
    mlflow.log_param("model_family", best_model_name)
    mlflow.log_param("selected_threshold", SELECTED_THRESHOLD)
    mlflow.log_param("threshold_criterion", "max F2 (out-of-fold CV, training set)")
    mlflow.log_param("n_features", len(FEATURE_COLUMNS))
    mlflow.log_params({f"hp_{k}": v for k, v in model_hyperparams.items()})

    # Metrics: the same final test-set numbers reported to stakeholders.
    mlflow.log_metrics(final_metrics)

    # Artifacts: the feature list, so a future consumer of this run can see
    # exactly what columns the pipeline expects without re-reading Notebook 4.
    feature_list_path = "logged_feature_columns.json"
    with open(feature_list_path, "w") as f:
        json.dump(FEATURE_COLUMNS, f, indent=2)
    mlflow.log_artifact(feature_list_path)
    os.remove(feature_list_path)

    # The model itself, registered under MODEL_NAME.
    model_info = mlflow.sklearn.log_model(
    sk_model=tuned_model,
    name="model",
    registered_model_name=MODEL_NAME,
    skops_trusted_types=["numpy.dtype"],
    )

print(f"Run ID:      {RUN_ID}")
print(f"Model URI:   {model_info.model_uri}")
```

```
Run ID:      e0c0cf5aedf9490487d9b67f84b4b5ac
Model URI:   models:/m-f8ab8501e6c64820ad5e3c92a2de5be0
```

```
Successfully registered model 'hospital-readmission-risk-classifier'.
Created version '1' of model 'hospital-readmission-risk-classifier'.
```

In [11]:
```
import mlflow
print(mlflow.__version__)
```

```
3.15.1
```

In [12]:
```
from mlflow import MlflowClient

client = MlflowClient()

client.transition_model_version_stage(
    name="hospital-readmission-risk-classifier",
    version="1",
    stage="Production",
)

print("Version 1 promoted to Production.")
```

```
Version 1 promoted to Production.
```

```
C:\Users\angie\AppData\Local\Temp\ipykernel_16260\1561414830.py:5: FutureWarning: ``mlflow.tracking.client.MlflowClient.transition_model_version_stage`` is deprecated since 2.9.0. Model registry stages will be removed in a future major release. To learn more about the deprecation of model registry stages, see our migration guide here: https://mlflow.org/docs/latest/model-registry.html#migrating-from-stages
  client.transition_model_version_stage(
```

In [13]:
```
version = client.get_model_version(
    name="hospital-readmission-risk-classifier",
    version="1",
)

print("Model:", version.name)
print("Version:", version.version)
print("Stage:", version.current_stage)
```

```
Model: hospital-readmission-risk-classifier
Version: 1
Stage: Production
```

7. Promote the New Version to "Production"

In [ ]:
```
# ---------------------------------------------------------------------
# 7.1 Find the registry version that was just created by this run
# ---------------------------------------------------------------------
client = MlflowClient()

matching_versions = [
    mv for mv in client.search_model_versions(f"name='{MODEL_NAME}'")
    if mv.run_id == RUN_ID
]
if not matching_versions:
    raise RuntimeError(
        f"Could not find a registered version of '{MODEL_NAME}' matching run {RUN_ID}."
    )
new_version = matching_versions[0].version

# ---------------------------------------------------------------------
# 7.2 Manually promote it, matching the proposal's promotion step
# ---------------------------------------------------------------------
# This is deliberately a manual, explicit step in Phase 1: a human decides
# this version is the one FastAPI should serve. Only Phase 2 (Airflow) would
# automate this decision, and only after an evaluation task confirms a
# challenger model actually beats whatever is currently tagged Production.
client.set_model_version_tag(
    name=MODEL_NAME,
    version=new_version,
    key="stage",
    value="Production",
)

print(f"Registered '{MODEL_NAME}' version {new_version} and tagged it stage=Production.")
```

8. Sanity Check: Reload the Production Model From the Registry

This confirms the round trip works end to end: a model version tagged "Production" can be found by querying the registry alone (no reference to `tuned_model` or `RUN_ID` from earlier in this notebook), loaded back, and produces the exact same predictions. This is the same lookup logic the FastAPI service in Section 9 will use.

In [ ]:
```
# ---------------------------------------------------------------------
# 8.1 Helper: find whichever version is currently tagged Production
# ---------------------------------------------------------------------
def get_production_version(client, model_name):
    """Return the highest-numbered model version tagged stage=Production.

    Highest version number is used as the tiebreaker in the rare case more
    than one version is left tagged Production, so the most recently
    promoted one always wins.
    """
    candidates = [
        mv for mv in client.search_model_versions(f"name='{model_name}'")
        if mv.tags.get("stage") == "Production"
    ]
    if not candidates:
        raise RuntimeError(
            f"No version of '{model_name}' is currently tagged stage=Production."
        )
    return max(candidates, key=lambda mv: int(mv.version))

# ---------------------------------------------------------------------
# 8.2 Reload it fresh from the registry and compare predictions
# ---------------------------------------------------------------------
prod_version = get_production_version(client, MODEL_NAME)
prod_model_uri = f"models:/{MODEL_NAME}/{prod_version.version}"
reloaded_model = mlflow.sklearn.load_model(prod_model_uri)
```

```
reloaded_probs = reloaded_model.predict_proba(X_test)[:, 1]
assert np.allclose(reloaded_probs, y_test_prob), (
    "Predictions from the reloaded Production model do not match the "
    "original tuned_model -- investigate before serving this version."
)

print(f"Reloaded Production model: version {prod_version.version} (run {prod_version.run_id})")
print("Predictions match the original fitted pipeline exactly.")
```

9. Build the FastAPI Serving App

The service always loads whichever model version is currently tagged `Production`, the same way Section 8 just verified. The classification threshold is read from that version's logged run parameters rather than hardcoded into the API, so promoting a new model version with a different tuned threshold requires no code change here -- this is the property the proposal describes as "FastAPI needs zero changes" once Phase 2 automated promotion is added.

The file is written to `serve/main.py` so it can be run as a normal FastAPI app (`uvicorn serve.main:app`) outside of this notebook, and is also imported and smoke-tested directly below in Section 10.

In [ ]:
```
# ---------------------------------------------------------------------
# 9.1 Write the FastAPI app to serve/main.py
# ---------------------------------------------------------------------
serve_dir = Path("serve")
serve_dir.mkdir(exist_ok=True)
(serve_dir / "__init__.py").touch(exist_ok=True)

# FEATURE_COLUMNS is embedded directly into the generated file so the API
# always matches whatever columns this notebook run actually trained on,
# without needing a separate config file kept in sync by hand.
app_source = f"""
'''
FastAPI serving app for the Hospital Readmission Risk Predictor.

Generated by 05_MLflow_Registry_FastAPI.ipynb -- do not hand-edit the
FEATURE_COLUMNS list below; re-run that notebook's Section 9 instead so it
stays in sync with whatever model is currently registered.

Run locally with:
    uvicorn serve.main:app --reload --port 8000
'''
import pandas as pd
from typing import Any, Dict, Optional

import mlflow
import mlflow.sklearn
from mlflow.tracking import MlflowClient
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

MLFLOW_TRACKING_URI = {MLFLOW_TRACKING_URI!r}
MODEL_NAME = {MODEL_NAME!r}
FEATURE_COLUMNS = {FEATURE_COLUMNS!r}

mlflow.set_tracking_uri(MLFLOW_TRACKING_URI)

app = FastAPI(
    title="Hospital Readmission Risk Predictor",
    description="Serves 30-day readmission risk scores from whichever model "
                "version is currently tagged Production in the MLflow registry.",
)

# Populated on startup by _load_production_model(); kept as simple module-level
# state so every request reuses the same loaded model instead of re-loading it.
_state: Dict[str, Any] = {{"model": None, "threshold": None, "version": None, "run_id": None}}


def _get_production_version(client: MlflowClient):
    '''Return the highest-numbered model version tagged stage=Production.'''
    candidates = [
        mv for mv in client.search_model_versions(f"name='{{MODEL_NAME}}'")
        if mv.tags.get("stage") == "Production"
    ]
    if not candidates:
        raise RuntimeError(f"No version of '{{MODEL_NAME}}' is tagged stage=Production.")
    return max(candidates, key=lambda mv: int(mv.version))


def _load_production_model():
```

```
    '''Load the current Production model and its tuned threshold, and cache both.'''
    client = MlflowClient()
    version = _get_production_version(client)
    model = mlflow.sklearn.load_model(f"models:/{{MODEL_NAME}}/{{version.version}}")

    # The threshold that was tuned specifically for this model version is
    # read from its own run, not hardcoded, so a future model version with
    # a different tuned threshold is served correctly with no code change.
    run = client.get_run(version.run_id)
    threshold = float(run.data.params["selected_threshold"])

    _state["model"] = model
    _state["threshold"] = threshold
    _state["version"] = version.version
    _state["run_id"] = version.run_id


@app.on_event("startup")
def startup_event():
    _load_production_model()


class PredictionRequest(BaseModel):
    '''One patient encounter. Keys should match the modeling feature names;
    missing keys are imputed by the model's own preprocessing pipeline,
    the same way missing values are handled during training.'''
    features: Dict[str, Any]


class PredictionResponse(BaseModel):
    readmission_probability: float
    predicted_high_risk: bool
    threshold_used: float
    model_version: str
    run_id: str


@app.get("/health")
def health():
    if _state["model"] is None:
        raise HTTPException(status_code=503, detail="Model not loaded.")
    return {{
        "status": "ok",
        "model_name": MODEL_NAME,
        "model_version": _state["version"],
        "run_id": _state["run_id"],
        "threshold": _state["threshold"],
    }}


@app.post("/predict", response_model=PredictionResponse)
def predict(request: PredictionRequest):
    if _state["model"] is None:
        raise HTTPException(status_code=503, detail="Model not loaded.")

    # Build a one-row DataFrame reindexed to the exact training column order.
    # Any feature not supplied in the request becomes NaN, which the
    # pipeline's SimpleImputer steps handle the same way they do at training time.
    row = pd.DataFrame([request.features]).reindex(columns=FEATURE_COLUMNS)

    probability = float(_state["model"].predict_proba(row)[:, 1][0])
    threshold = _state["threshold"]

    return PredictionResponse(
        readmission_probability=probability,
        predicted_high_risk=probability >= threshold,
        threshold_used=threshold,
        model_version=str(_state["version"]),
        run_id=_state["run_id"],
    )


if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
"""

(serve_dir / "main.py").write_text(app_source)
print(f"Wrote FastAPI app to {(serve_dir / 'main.py').resolve()}")
```

10. Smoke-Test the API In-Notebook

`TestClient` runs the FastAPI app in-process, including its startup event (which loads the Production model), without needing a separately running `uvicorn` server. This confirms the endpoint works end to end before anyone tries to run it for real.

In [ ]:
```python
# ---------------------------------------------------------------------
# 10.1 Import the freshly written app and start it via TestClient
# ---------------------------------------------------------------------
import sys
import importlib
import json
import math
import numpy as np

sys.path.insert(0, ".")
import serve.main as serve_app
importlib.reload(serve_app)  # picks up the file just written, if this cell is re-run

with TestClient(serve_app.app) as client:

    # -----------------------------------------------------------------
    # 10.2 Health check
    # -----------------------------------------------------------------
    health_response = client.get("/health")

    print("GET /health ->", health_response.status_code)
    print(json.dumps(health_response.json(), indent=2))

    # -----------------------------------------------------------------
    # 10.3 A real prediction, using one actual test-set encounter
    # -----------------------------------------------------------------
    sample_encounter = X_test.iloc[0].to_dict()

    # Convert NumPy scalar values to native Python values.
    sample_encounter = {
        k: (v.item() if hasattr(v, "item") else v)
        for k, v in sample_encounter.items()
    }

    # JSON does not allow NaN, +Infinity, or -Infinity.
    # Convert those values to None (JSON null).
    for k, v in sample_encounter.items():
        if isinstance(v, float) and not math.isfinite(v):
            sample_encounter[k] = None

    # Display any values that were converted.
    converted_missing = [
        k for k, v in sample_encounter.items() if v is None
    ]

    if converted_missing:
        print("\nNon-finite/missing values converted to JSON null:")
        for column in converted_missing:
            print(f"  - {column}")

    # -----------------------------------------------------------------
    # Send the prediction request
    # -----------------------------------------------------------------
    predict_response = client.post(
        "/predict",
        json={"features": sample_encounter}
    )

    print("\nPOST /predict ->", predict_response.status_code)

    # Print response safely, including useful error information if
    # the FastAPI endpoint itself rejects the request.
    try:
        print(json.dumps(predict_response.json(), indent=2))
    except Exception:
        print(predict_response.text)

    # -----------------------------------------------------------------
    # 10.4 Compare prediction with the actual test-set label
    # -----------------------------------------------------------------
    print(
        f"\nActual label for this encounter (readmitted_30d): "
        f"{int(y_test.iloc[0])}"
    )
```

11. How to Run This Service

Outside of this notebook, from the project root:

```
uvicorn serve.main:app --reload --port 8000
```

Then, for example:

```
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"features": {"time_in_hospital": 5, "num_medications": 12, "age": "[70-80)"}}'
```

Any features left out of the request are imputed by the pipeline's own preprocessing steps, the same way missing values are handled at training time, so a caller does not have to supply every modeling column on every request.

This is a local, development-grade demonstration of the serving layer, consistent with what Phase 1 of the proposal calls for. It is not a production deployment: there is no authentication, no request logging or monitoring, and no containerization. Those would need to be added before this endpoint was exposed to a real hospital system, and are explicitly out of scope for this capstone phase.

Next Steps

1. Route the F1-vs-F2 threshold decision to the care-management stakeholder audience (carried over, unresolved, from the Hyperparameter Tuning & Model Evaluation report); update `selected_threshold` for the next registered version if that decision changes it.
2. If Phase 2 is pursued: wrap the Notebook 4 training logic in an Airflow DAG that retrains on a schedule, evaluates the challenger against whatever version is currently tagged Production using the same metrics logged in Section 6 here, and only re-tags a new version Production if it actually wins. No changes to `serve/main.py` would be required for this, since it already always looks up whichever version is tagged Production at request time.
3. Before any real deployment: add authentication to the FastAPI endpoint, request/response logging for monitoring drift, and containerize the service.

End of Notebook 5

Primary deliverables: a tracked, registered MLflow run for the selected tuned classifier (parameters, test metrics, and the fitted pipeline artifact), a version manually promoted to "Production" in the Model Registry, and a working FastAPI `/predict` endpoint that serves live predictions from whichever version holds that tag.

GitHub: https://github.com/AngelineSetiawan/AngelineSetiawan-hospital-readmission-risk-predictor